NLP Laboratory — Assignment 7

Hindi Part-of-Speech Tagging: Naive Bayes and HMM (Forward, Backward and Forward-Backward Viterbi)

Name	GAURAV DAMOR	**Course**	Natural Language Processing
Roll No.	U24AI026	**Lab**	Lab 7 — Hindi POS Tagging

325,710 Training tokens	**24** POS tags (BIS tagset)	**94.37%** Best test accuracy (Fwd Viterbi)	**4.59%** Test OOV rate

1. Task and Data

The goal is to assign a part-of-speech tag to every token of a Hindi sentence. The corpus is supplied as three tab-separated files (*word, tag* per line, blank line between sentences) using the BIS tagset. All probabilities are estimated **only from the training split**. The development split is used to choose the two smoothing constants, and the test split is used once, for final reporting. Four tagging algorithms are compared: a token-level **Naive Bayes** classifier and a bigram **Hidden Markov Model** decoded with **forward Viterbi**, **backward Viterbi** and **forward-backward Viterbi**.

Split	Sentences	Tokens	OOV rate (w.r.t. train vocabulary)
Train	15,087	325,710	— (vocabulary = 18,279 types)
Dev	1,866	40,762	4.19%
Test	1,910	41,415	4.59%

Table 1: Corpus statistics.

2. Methods and Mathematical Formulation

2.1 Naive Bayes classifier

Each token is classified on its own using the five features required by the assignment: **length** (MORE if the word has more than 4 characters, else LESS), the **word**, the **previous word**, the **next word** and the **previous tag**. Sentence boundaries are represented by <s> and </s>. Features are assumed conditionally independent given the tag:

```
t* = argmax_t [ log P(t) + Σ_k log P(f_k | t) ]
P(f_k = v | t) = (C_k(t, v) + α) / (C(t) + α(|V_k| + 1)), α = 0.1 (tuned on dev)
```

At test time the gold previous tag is unavailable, so the sentence is decoded **greedily from left to right** and the previous-tag feature takes the classifier's own prediction for the preceding token.

2.2 HMM parameter estimation

Each sentence is wrapped as <s> t_1 … t_n </s> and tag-bigram counts $C(t_i, t_j)$ are collected. Transitions are add-k smoothed. Emissions use relative frequencies, and each tag keeps back a share of probability for unseen words. That share is proportional to H_t, the number of hapax-legomena word types (types seen exactly once) carrying tag t. This is a Good-Turing-style estimate, so open classes such as N_NNP and JJ receive far more unknown-word mass than closed classes such as PSP.

```
A_f(i→j) = P(t_k=j | t_(k-1)=i) = (C(i,j) + k) / (C(i) + k(T+1)), k = 0.1 (tuned on dev)
A_b(i→j) = P(t_k=j | t_(k+1)=i) = (C(j,i) + k) / (C(i) + k(T+1))
P(w | t) = C(t, w) / (C(t) + H_t) P(UNK | t) = H_t / (C(t) + H_t)
```

2.3 Forward (standard) Viterbi

```
δ_1(j) = log A_f(<s>→j) + log P(w_1|j)
δ_k(j) = max_i [ δ_(k-1)(i) + log A_f(i→j) ] + log P(w_k|j), ψ_k(j) = argmax_i [...]
t_n = argmax_j [ δ_n(j) + log A_f(j→</s>) ], then follow ψ backwards to t_1.
```

2.4 Backward Viterbi

The recursion runs from right to left over the reversed chain. It starts at </s>, uses the backward transitions A_b, ends at <s>, and then follows the pointers forwards:

```
δ^b_n(j) = log A_b(</s>→j) + log P(w_n|j)
δ^b_k(j) = max_i [ δ^b_(k+1)(i) + log A_b(i→j) ] + log P(w_k|j)
t_1 = argmax_j [ δ^b_1(j) + log A_b(j→<s>) ], then follow the pointers forwards to t_n.
```

2.5 Forward-backward Viterbi

A full forward pass (α) and a full backward pass (β) are run over the same trellis, and every position is labelled with the tag of highest posterior probability. Viterbi returns the single most probable *sequence*; this decoder instead maximises the expected number of correctly tagged *tokens*, which is what per-tag F1 and accuracy measure.

```
α_k(j) = P(w_1..w_k, t_k=j) = Σ_i α_(k-1)(i) A_f(i→j) P(w_k|j)
β_k(i) = P(w_(k+1)..w_n | t_k=i) = Σ_j A_f(i→j) P(w_(k+1)|j) β_(k+1)(j)
t_k* = argmax_j α_k(j) · β_k(j) ( ∝ P(t_k = j | w_1..w_n) )
```

Correctness check. Replacing the sums with max (max-product forward × backward trellises) must reproduce the forward-Viterbi path. On both splits it agrees on 100.00% of tokens.

3. Results

3.1 Overall performance

Algorithm	Split	Accuracy	Macro F1	Weighted F1	Known-word acc.	OOV-word acc.
Naive Bayes	DEV	92.47	83.31	92.49	94.27	51.14
	TEST	92.57	84.44	92.59	94.34	55.77
Fwd Viterbi	DEV	94.36	94.65	94.38	**95.95**	58.11
	TEST	**94.37**	91.16	**94.38**	**95.97**	**61.08**
Bwd Viterbi	DEV	94.36	94.65	94.38	95.95	58.11
	TEST	94.37	91.16	94.38	95.97	61.08
Fwd-Bwd Viterbi	DEV	**94.39**	**94.70**	**94.40**	95.95	**58.70**
	TEST	94.31	**91.17**	94.32	95.93	60.51

Table 2: Overall scores (%). Green/bold marks the best value per split and metric. OOV = word not seen in training.

3.2 Per-tag F1-score

			DEV F1				TEST F1			
Tag	Meaning	Test n	NB	Fwd	Bwd	Fwd-Bwd	NB	Fwd	Bwd	Fwd-Bwd
N_NN	common noun	9303	0.907	0.919	0.919	**0.919**	0.901	**0.917**	**0.917**	0.915
PSP	postposition	8245	0.985	0.991	0.991	**0.991**	0.982	0.990	0.990	**0.990**
N_NNP	proper noun	5296	0.836	**0.858**	**0.858**	0.857	0.841	**0.867**	**0.867**	0.864
V_VM	main verb	4174	0.959	0.960	0.960	**0.960**	**0.963**	0.961	0.961	0.961
V_VAUX	auxiliary verb	2534	**0.974**	0.970	0.970	0.969	**0.976**	0.972	0.972	0.971
RD_PUNC	punctuation	2549	0.993	**1.000**	**1.000**	**1.000**	0.994	**1.000**	**1.000**	**1.000**
JJ	adjective	2258	0.869	0.884	0.884	**0.886**	0.887	**0.892**	**0.892**	0.892
PR_PRP	personal pronoun	1338	0.897	**0.973**	**0.973**	0.973	0.907	**0.973**	**0.973**	0.973
QT_QTC	cardinal number	874	0.917	0.914	0.914	**0.920**	**0.920**	0.892	0.892	0.894

			DEV F1				TEST F1			
Tag	Meaning	Test n	NB	Fwd	Bwd	Fwd-Bwd	NB	Fwd	Bwd	Fwd-Bwd
CC_CCD	coordinating conj.	772	0.948	**0.980**	**0.980**	0.979	0.962	0.976	0.976	**0.976**
CC_CCS	subordinating conj.	736	0.968	**0.986**	**0.986**	**0.986**	0.967	0.985	0.985	**0.986**
N_NST	spatio-temporal noun	625	0.890	0.982	0.982	**0.984**	0.889	0.978	0.978	**0.980**
RP_RPD	default particle	617	0.885	**0.949**	**0.949**	0.948	0.883	**0.942**	**0.942**	0.939
DM_DMD	deictic demonstrative	516	0.883	0.956	0.956	**0.957**	0.898	**0.961**	**0.961**	0.958
RD_SYM	symbol	464	0.919	**1.000**	**1.000**	**1.000**	0.943	**0.999**	**0.999**	**0.999**
QT_QTF	general quantifier	323	0.864	**0.938**	**0.938**	0.934	0.856	0.952	0.952	**0.954**
RP_NEG	negation particle	243	0.904	**0.994**	**0.994**	**0.994**	0.888	**0.994**	**0.994**	**0.994**
RB	adverb	167	0.612	0.830	0.830	**0.834**	0.661	**0.873**	**0.873**	**0.873**
PR_PRF	reflexive pronoun	151	0.916	**1.000**	**1.000**	**1.000**	0.903	**0.997**	**0.997**	**0.997**
PR_PRL	relative pronoun	91	0.728	0.947	0.947	**0.952**	0.725	**0.973**	**0.973**	**0.973**
QT_QTO	ordinal number	78	0.601	**0.899**	**0.899**	**0.899**	0.667	0.898	0.898	**0.904**
RP_INTF	intensifier	29	0.758	**0.785**	**0.785**	**0.785**	0.821	**0.889**	**0.889**	**0.889**
PR_PRQ	wh-pronoun	31	0.783	**1.000**	**1.000**	**1.000**	0.830	**1.000**	**1.000**	**1.000**
RP_INJ	interjection	1	0.000	**1.000**	**1.000**	**1.000**	0.000	0.000	0.000	0.000

Table 3: Per-tag F1 on dev and test, tags ordered by training frequency. NB = Naive Bayes; Fwd, Bwd, Fwd-Bwd = HMM forward, backward and forward-backward Viterbi. The best algorithm for each tag and split is highlighted (ties are all highlighted).

4. Charts

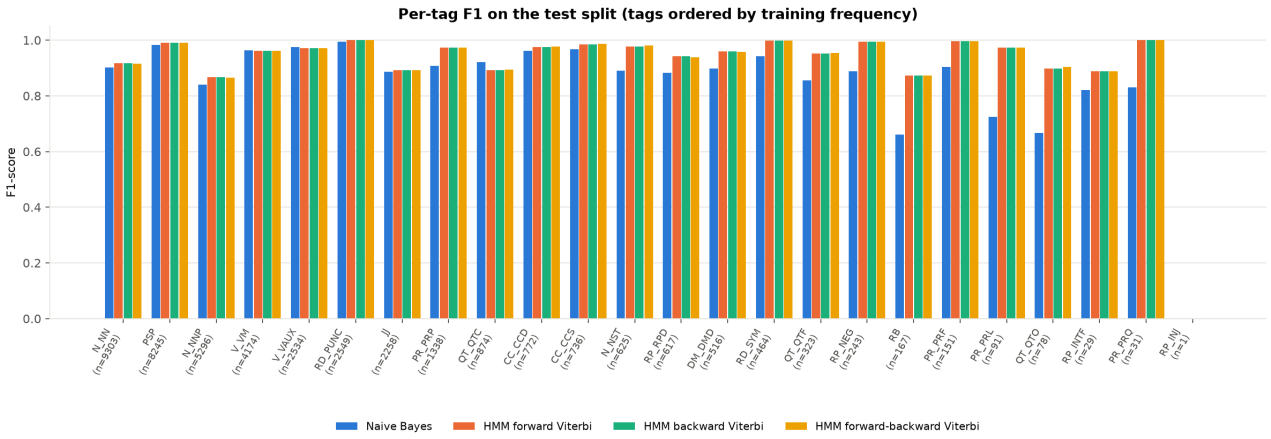


Figure 1: Per-tag F1 on the test split for all four algorithms.

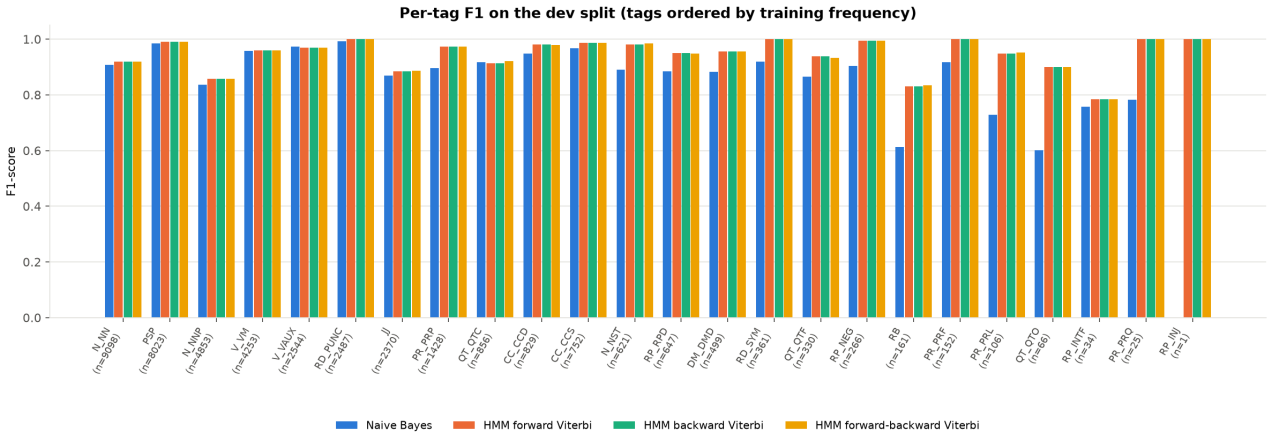


Figure 2: Per-tag F1 on the dev split for all four algorithms.

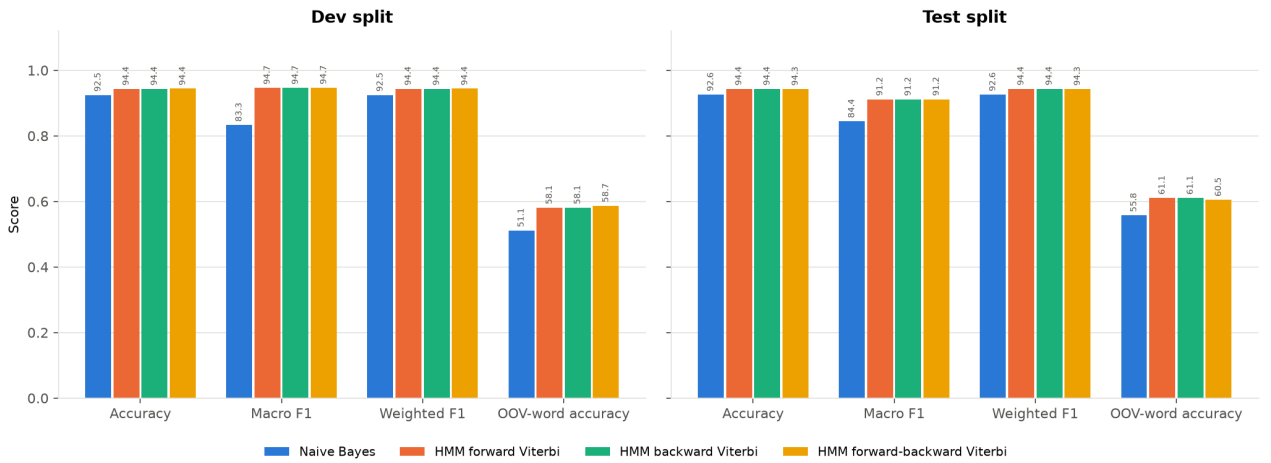


Figure 3: Accuracy, macro-F1, weighted-F1 and OOV-word accuracy on dev and test.

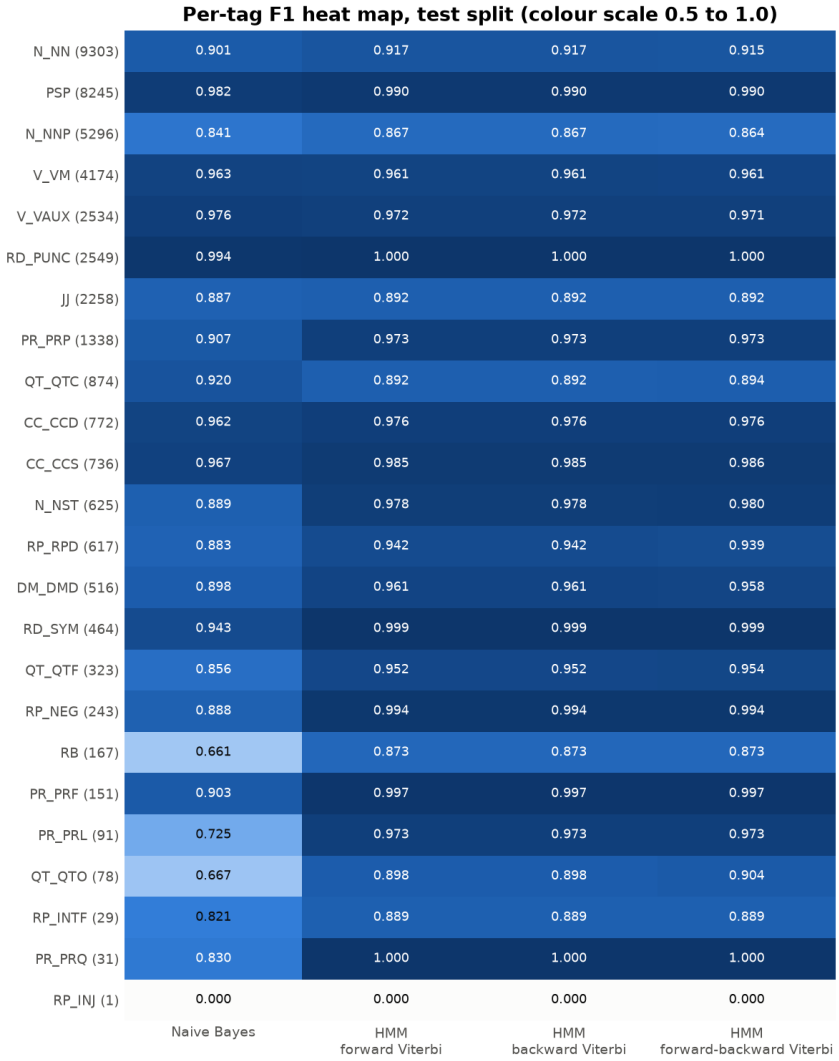


Figure 4: Per-tag F1 heat map (test). Darker = higher F1.

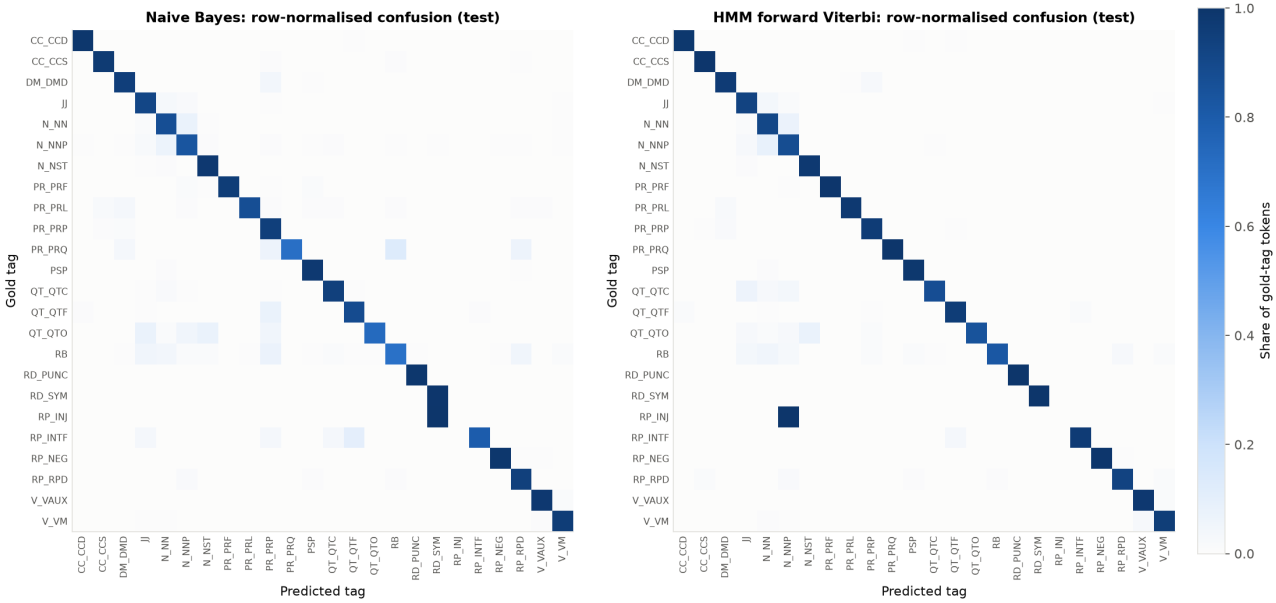


Figure 5: Row-normalised confusion matrices on test: Naive Bayes (left) vs the best HMM decoder (right).

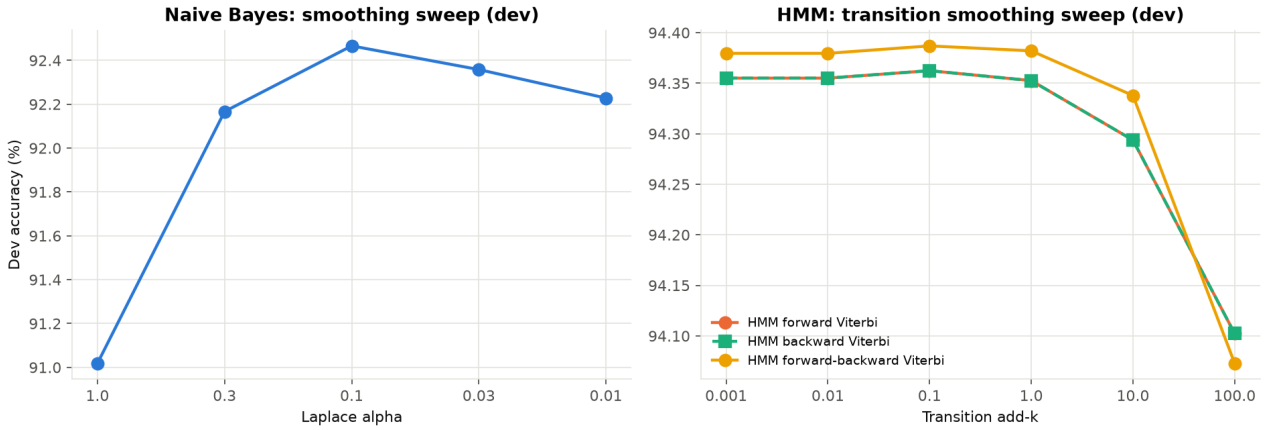


Figure 6: Dev-set sweeps used to choose the smoothing constants. In the HMM panel the backward-Viterbi curve sits exactly on top of the forward-Viterbi curve.

5. Analysis and Discussion

5.1 HMM vs Naive Bayes. The HMM taggers beat Naive Bayes on test by 1.80 accuracy points (94.37% vs 92.57%) and by 6.7 macro-F1 points. The gap is largest on low-frequency closed-class tags: RB, PR_PRL, QT_QTO, PR_PRQ, RP_NEG and N_NST. Naive Bayes has high *recall* but poor *precision* on these tags (for example N_NST precision 0.81, PR_PRL 0.62, RP_NEG 0.80). The cause is Laplace smoothing on the open-vocabulary context features. An unseen previous/next word costs $\log(\alpha / (C(t) + \alpha|V|))$, and this penalty is *smaller* for a rare tag (small C(t)) than for a frequent one. When the context words are new, rare tags are therefore pulled in. The HMM conditions only on the neighbouring *tags* (24 values, not tens of thousands of words) and scores the whole sentence jointly, so it avoids this bias.

5.2 Effect of smoothing and of the previous-tag feature in Naive Bayes. Plain add-one ($\alpha = 1$) gives only 91.02% dev accuracy, while the tuned $\alpha = 0.1$ gives 92.47%. Add-one gives too much mass to unseen word values, which is the same over-smoothing effect seen in the n-gram labs. Supplying the *gold* previous tag instead of the predicted one raises test accuracy only from 92.57% to 93.15%, so greedy error propagation costs about 0.6 points. Most of the gap to the HMM comes from the independence assumption itself rather than from that feature.

5.3 Why forward and backward Viterbi give identical output. The two decoders produced exactly the same tag for every token on both splits, and for every k in the sweep. This is expected, not a bug. On the augmented chain <s> $t_1 \ldots t_n$ </s> every tag occurrence has exactly one predecessor and one successor, so the row sum and column sum of the bigram matrix both equal C(t), and both boundary symbols occur once per sentence. Hence:

```
Π_k A_f(t_(k-1)→t_k) = Π_k (C(t_(k-1),t_k)+k) / Π_k (C(t_(k-1))+k(T+1))
Π_k A_b(t_(k+1)→t_k) = Π_k (C(t_k,t_(k+1))+k) / Π_k (C(t_(k+1))+k(T+1))
The numerators run over the same bigrams, and the denominators run over the same multiset
{<s>/</s>, t_1, …, t_n}.
```

The left-to-right and right-to-left HMMs therefore define **the same joint distribution** P(tags, words), so their argmax sequences coincide. Numerically, the largest absolute difference in log P(tags, words) over all dev sentences is 2.3e-13, which is floating-point rounding. Running Viterbi in the other direction changes only the order of the dynamic program, not the model, so it cannot change the answer.

5.4 Forward-backward (posterior) decoding. Posterior decoding changed 0.44% of test tokens relative to Viterbi. Accuracy moved by -0.06 points on test and +0.02 on dev, and OOV accuracy moved by -0.58 points on test. Posterior decoding is optimal for expected token accuracy *under the model*. With a well-peaked bigram HMM the per-token posteriors usually agree with the best path, so in practice the two decoders are statistically tied. Its gains are concentrated in ambiguous regions such as runs of unknown words.

5.5 Error analysis. The most frequent HMM errors on test are N_NN→N_NNP (656), N_NNP→N_NN (439), N_NNP→JJ (142), N_NN→JJ (115). Separating common from proper nouns is the hardest decision in Hindi: there is no capitalisation, and many names (person, place and organisation names) are also ordinary words. The noun/adjective boundary is also fuzzy. Unknown words are the main weakness of every model: OOV accuracy is only 61.08% (HMM) and 55.77% (NB), against about 95.97% for known words, because the only signal for an unseen word is its context. Suffix-based emission models for unknown words (Hindi inflectional endings for verbs, plurals and

adjectives) would be the most effective next step.

5.6 A note on macro-F1. Test macro-F1 for the HMM (91.16%) is lower than on dev (94.65%) mainly because the test split contains a single RP_INJ (interjection) token, a tag with only 11 training occurrences. Missing that one token gives the tag an F1 of 0 and pulls the 24-tag average down by about 4 points. Weighted F1 and accuracy, which reflect token frequency, are essentially identical on dev and test.

6. Conclusion

A bigram HMM decoded with Viterbi tags Hindi text with 94.37% test accuracy, 1.8 points above a carefully smoothed Naive Bayes classifier that uses word, context-word, length and previous-tag features. Modelling the tag *sequence* jointly matters most for rare closed-class tags. Backward Viterbi is mathematically equivalent to forward Viterbi for a bigram HMM estimated from counts, and the experiments confirm this exactly. Forward-backward posterior decoding is statistically tied with Viterbi. The remaining errors are dominated by the N_NN/N_NNP distinction and by out-of-vocabulary words.